Lazy Initialization
Builder

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 13

Categorizing Design Patterns

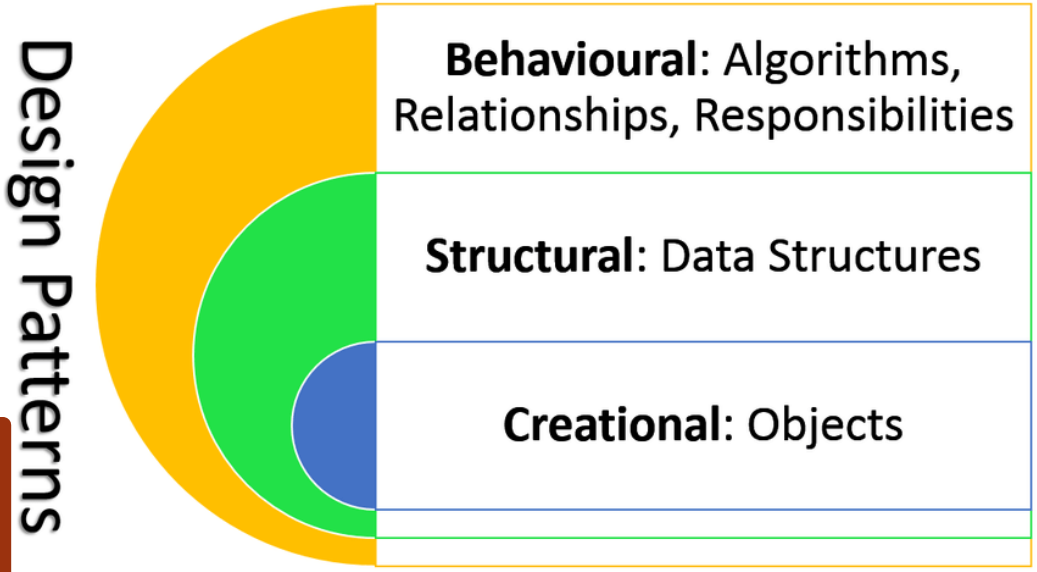
- **Behavioural**

Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural** How do classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

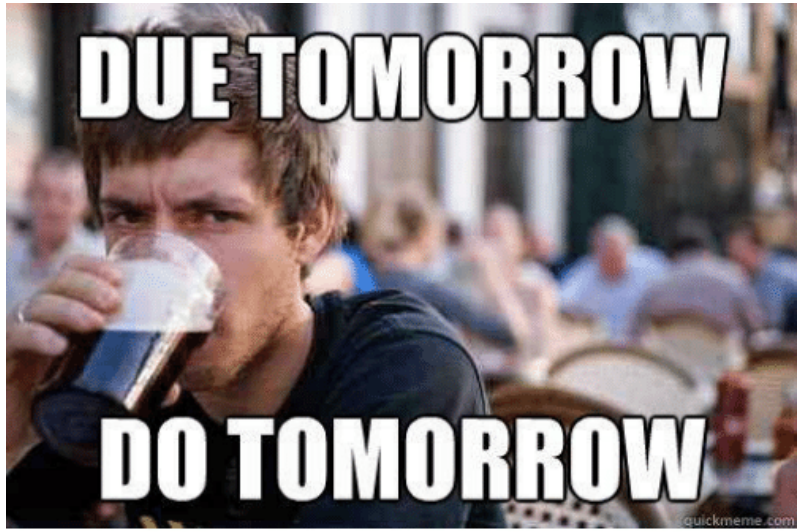
- **Creational (We are looking at these!)**

All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects



Lazy Initialization

Lazy Initialization



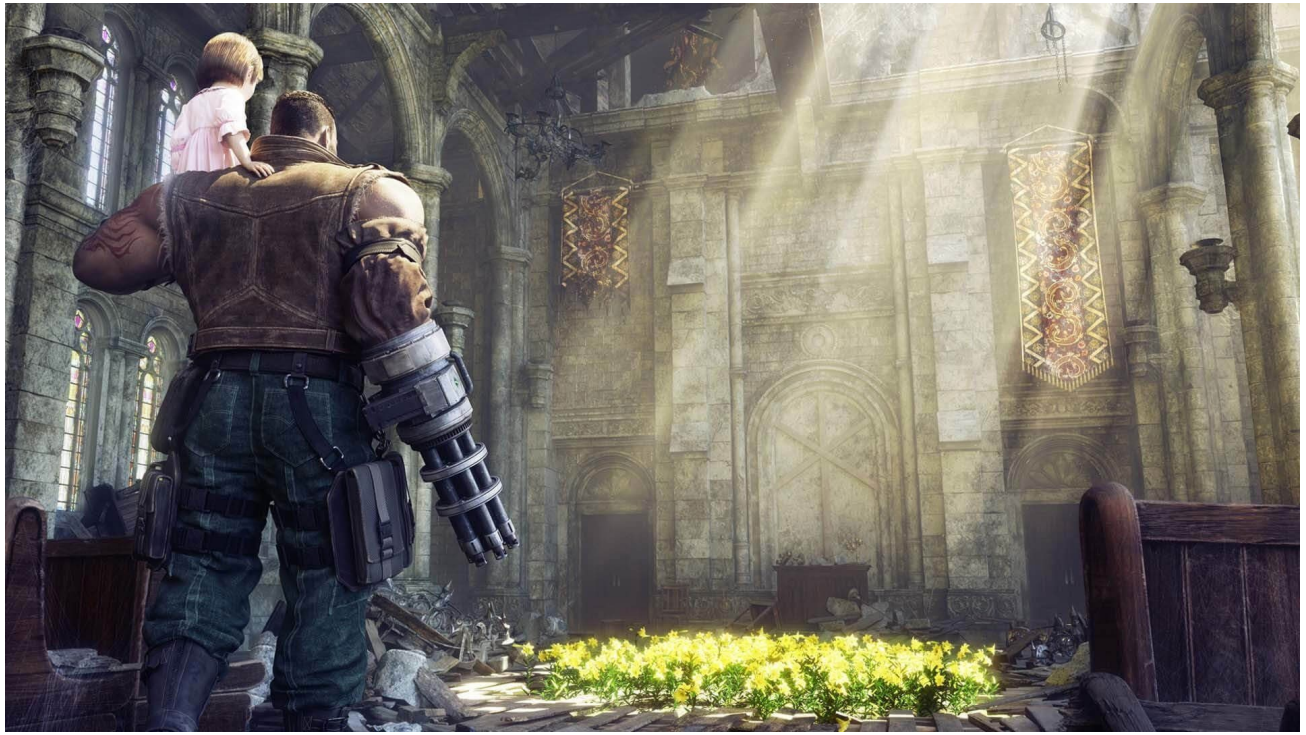
The art of procrastination.

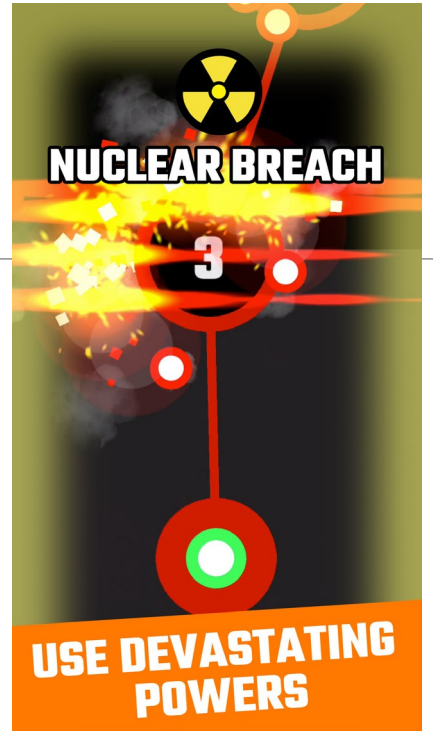
- **Lazy Initialization** is probably the easiest design pattern out there. It is also known as **Lazy Loading**.
- The Lazy Initialization pattern mandates that we **don't initialize an object/resource unless we need it.**
- Why tax the system unnecessarily?
- This is usually applied to objects that are **expensive to initialize** either due to memory or computation costs.
- In simple words. We **procrastinate on initializing an expensive resource until we need it**. Laziness is the hallmark of every good programming.

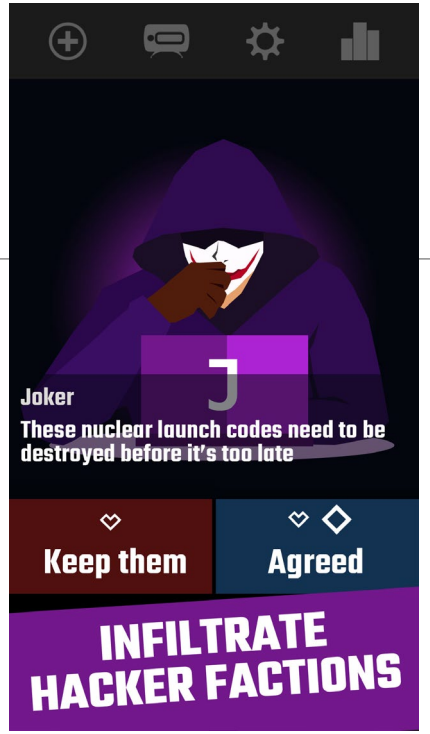
Examples

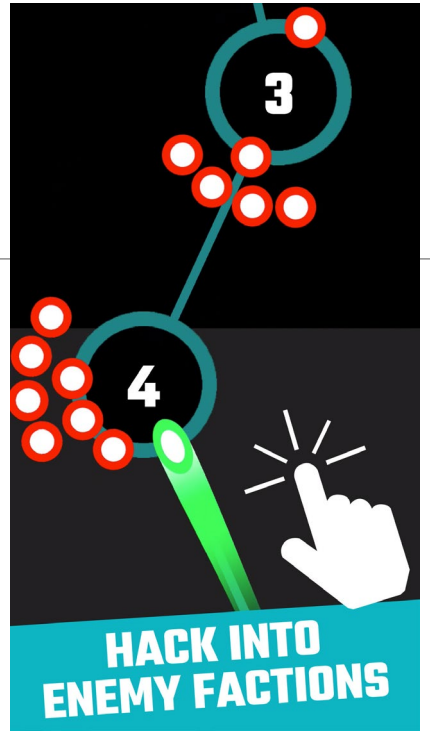
Lazy Initialization is used when rendering complex environments in games for example.

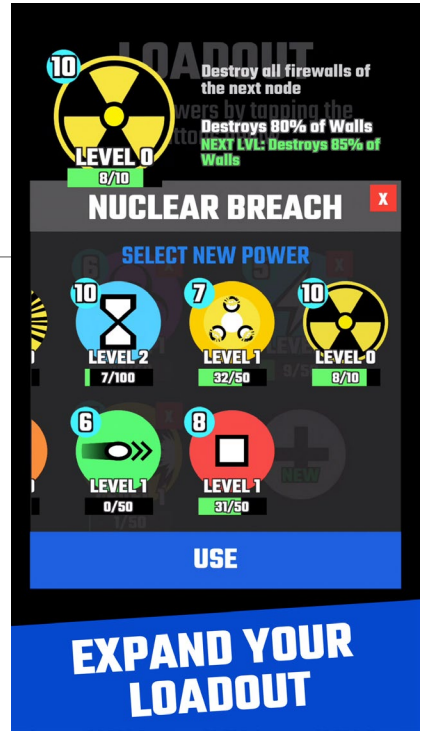
If the player is not looking at an area, don’t keep it in memory and render it.









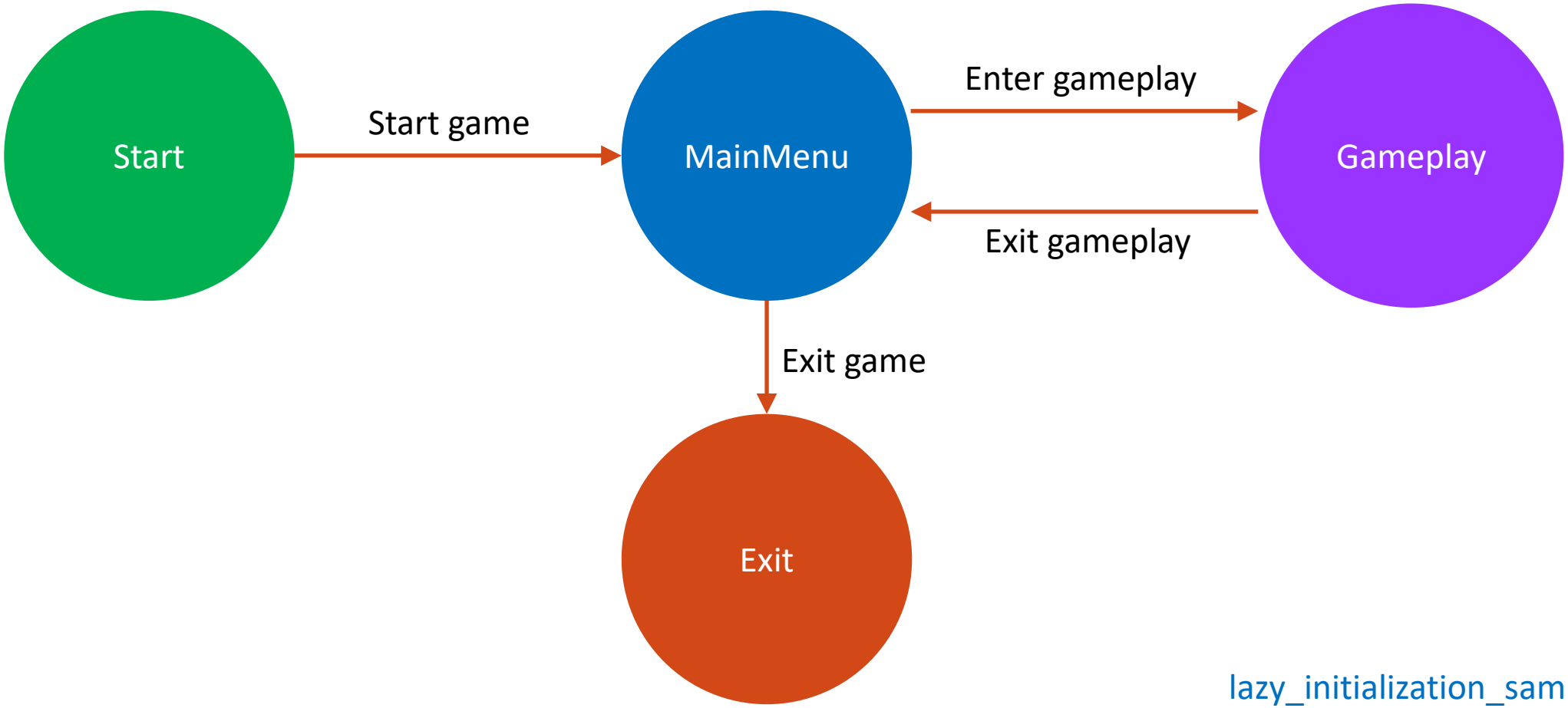


I've used lazy initialization in my games. Games have many screens with UI components. The player won't always see every screen when they play the game

Don't load and save all screens into memory when the game starts

Load screens when the player needs to access them

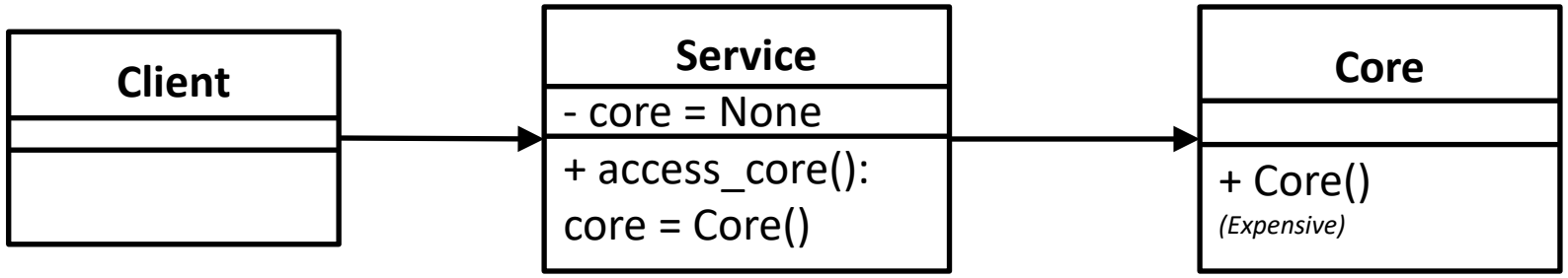
Menu transition state machine



lazy_initialization_sample_code.py

Lazy Initialization

- That's it.
- That's lazy initialization.
- Let's look at its advantages and disadvantages.
- Also, it can be implemented in so many ways, there really isn't much point in a UML diagram.
- But if you insist. Here is a possible UML diagram.



Lazy Initialization

Advantages:

- Dedicate resources (memory / processing) on a on-demand basis.
- Speed up execution when not accessing expensive resource

Disadvantages:

- Not a good choice if the object needs to be initialized every time it is accessed (assuming it is accessed frequently).

Builder

WHEN YOU WANT CUSTOMIZATION AND COMPLEXITY

Builder

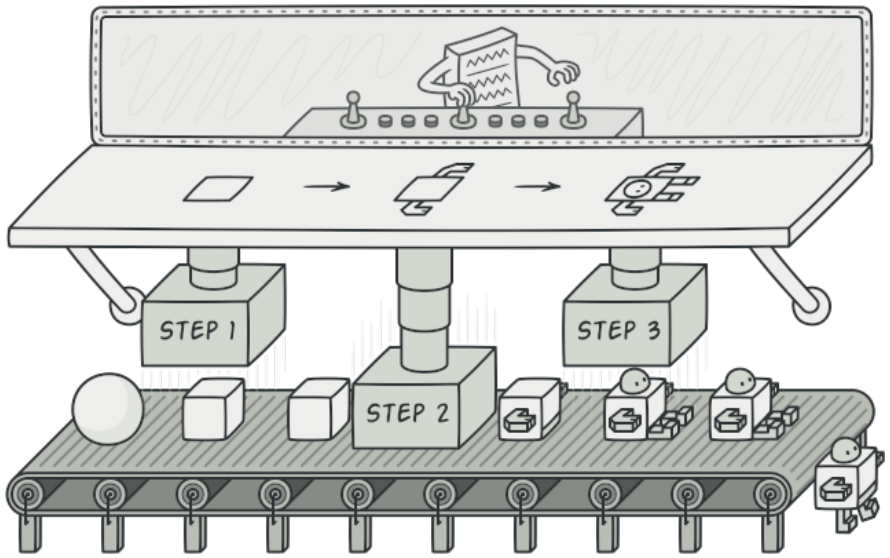
Often we need to initialize objects with a number of different parts, nested fields, optional attributes and behaviour.

This can cause either:

- The initializer of an object to become **bloated**. **Lots of parameters**, most of which may not even be used often.
  - `def __init__(param1, param2, param3 … paramX):`
- A complex inheritance hierarchy to model the optional features.

The **Builder Pattern** allows us to **break this construction code into a separate class**.

This separate class **decouples independent blocks of construction code** that can be **mixed and matched**.



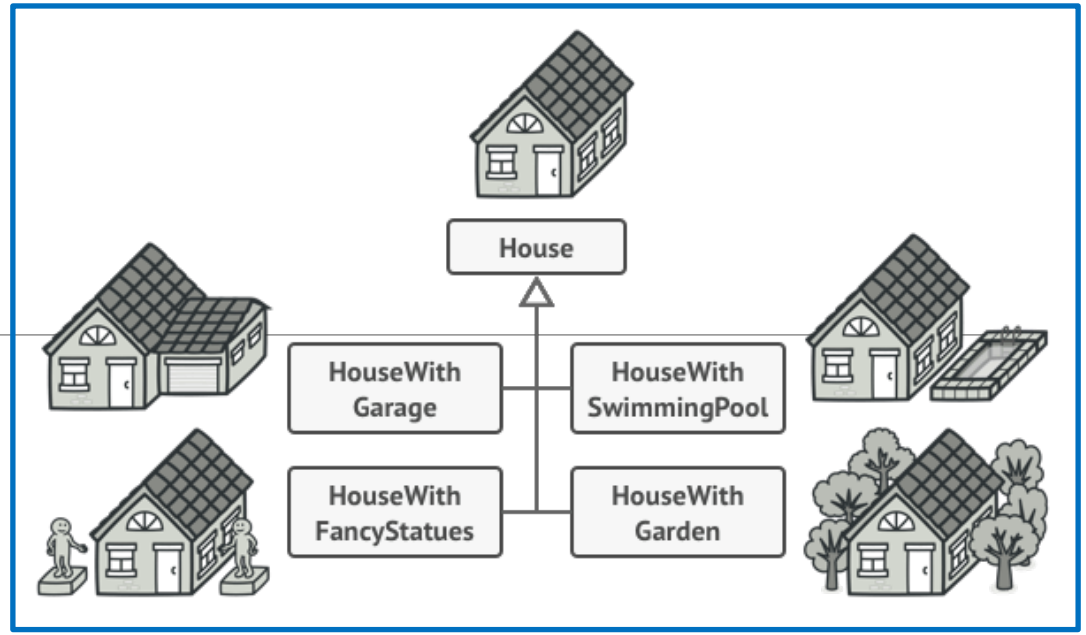
Builder

Say we were modeling and representing Houses.

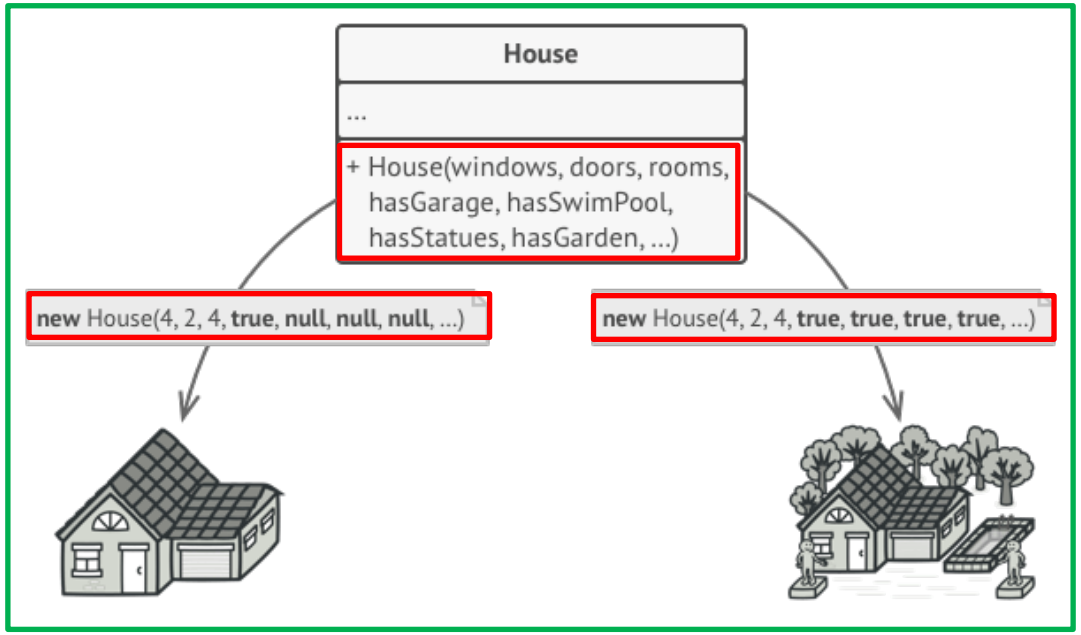
Houses have a lot of **components and optional parts**.

- Door, walls, roof, windows, etc.

We can use **inheritance** to model all these cases.



Or we can avoid inheritance, and have a **single class with a lot of parameters** in its initializer and unnecessary attributes to keep track of all the components. This leads to **ugly initializer calls**.



Builder

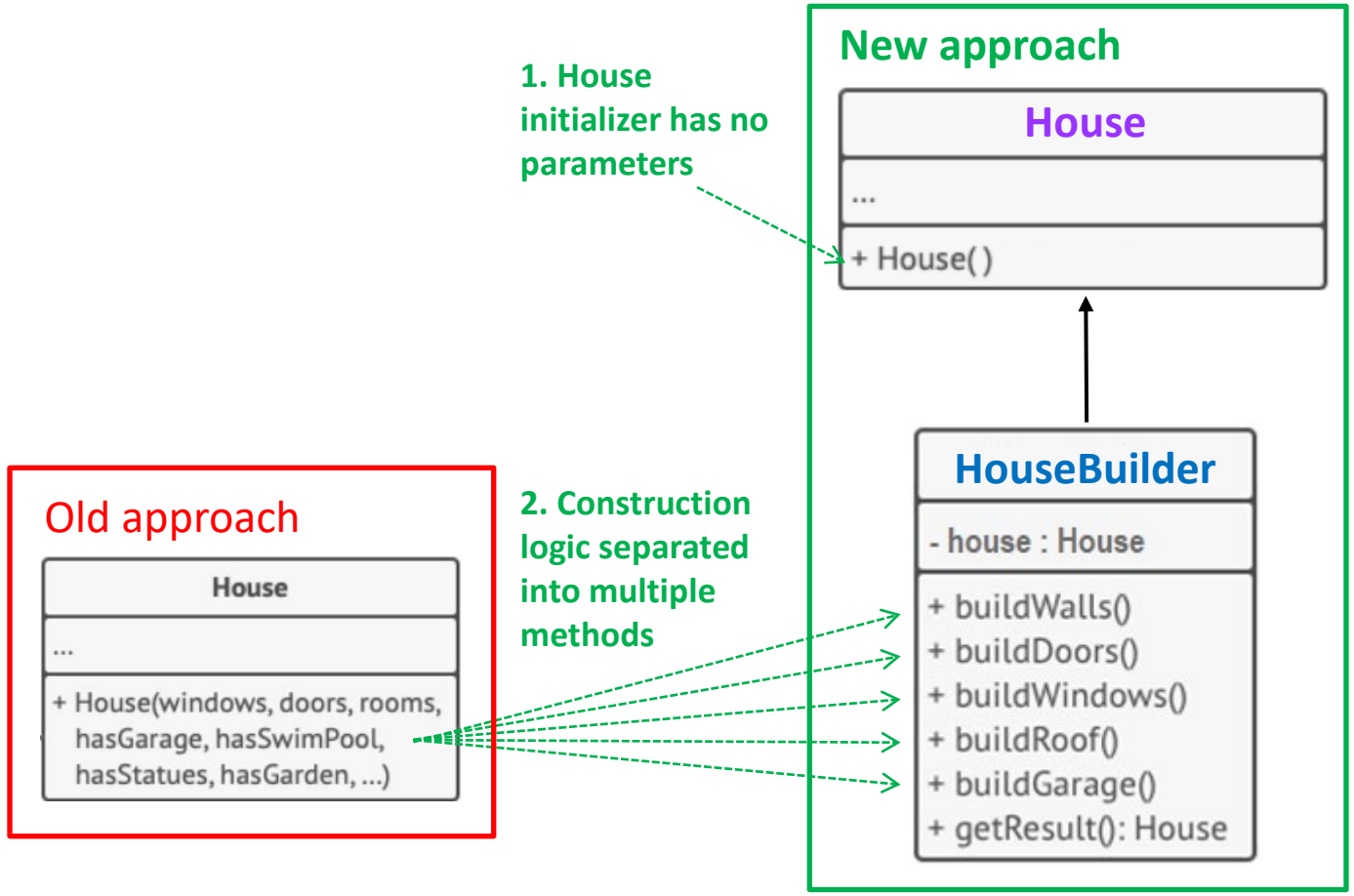
If the construction of an object is so complex, then why don't we separate that out into its own class (**HouseBuilder**).

This would ensure the **Single Responsibility Principle**.

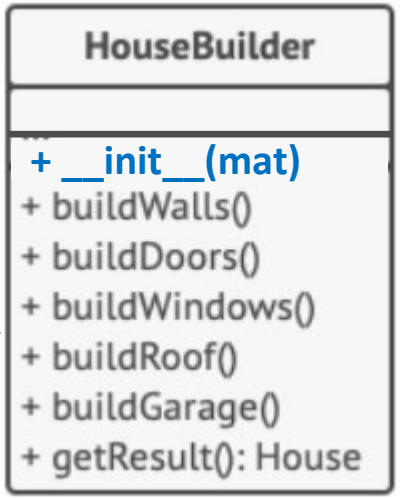
A class would be split into itself (**House**) and its builder class (**HouseBuilder**) with all of its construction code.

We can have separate calls for each different component that our object is aggregated/composed of.

Calling different combinations of these methods would lead to different types of homes.



Builders and Variations



This is a really powerful pattern.

We can customize the Builder Class to take in its own parameters and options.

This can allow us to instantiate different Builder objects that execute the construction differently.

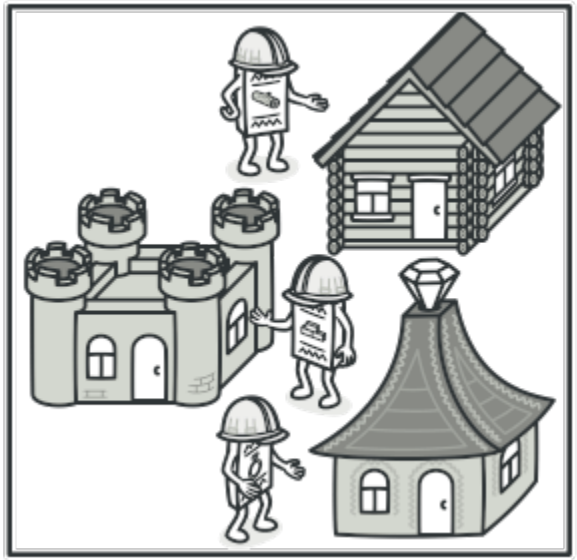
For example, we can have our builder class accept a parameter for the material we can build the house out of.

We can then make 3 builder objects:

```
builder_wood = HouseBuilder(mat="Wood")
cabin = builder_wood.build_house()

builder_stone = HouseBuilder(mat="Stone")
castle = builder_stone.build_house()

builder_paper = Housebuilder(mat="Paper")
origami_house = builder_paper.build_house()
```



Builder and Director

We can **optionally** implement a manager class called a **Director** that can be responsible for different **subroutines** that call different methods in a builder class.

This Director class is optional and usually makes sense when working with complex objects that can be decoupled into different pieces/components that can be mixed and matched.

We **can even inherit** from a Builder class to create different variations of builders if the complexity demands it.

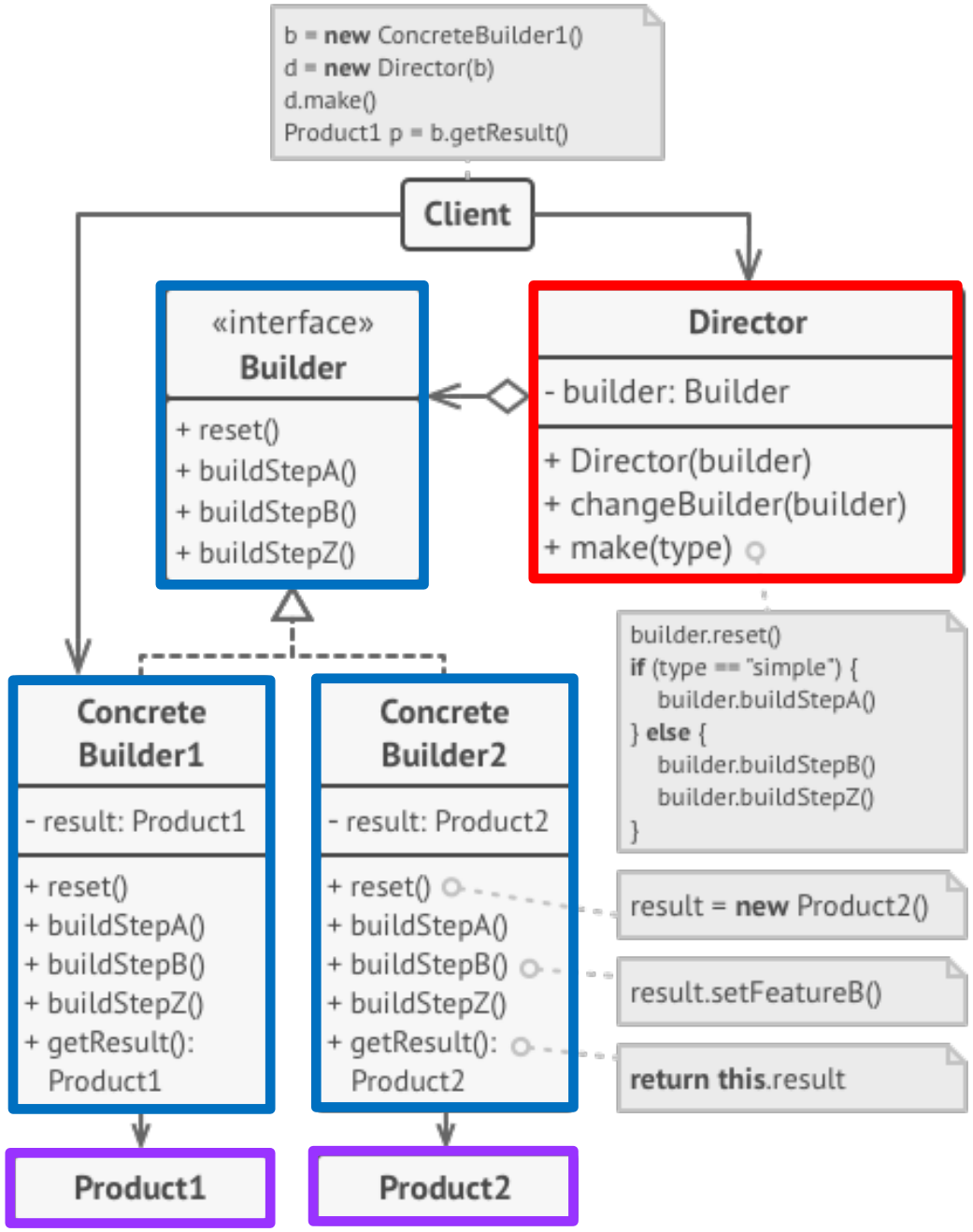
```
class Director:

    def __init__(self, builder: Builder):
        self.builder = builder

    def build_townhouse(self) -> House:
        self.builder.build_walls()
        self.builder.build_roof()
        self.builder.build_garage()
        self.builder.build_swimming_pool()
        self.builder.build_second_story()
        self.builder.build_door()
        self.builder.build_windows()
        return self.builder.product

    def build_apartment(self) -> House:
        self.builder.build_walls()
        self.builder.build_door()
        self.builder.build_windows()
        return self.builder.product
```

Builder Pattern: UML

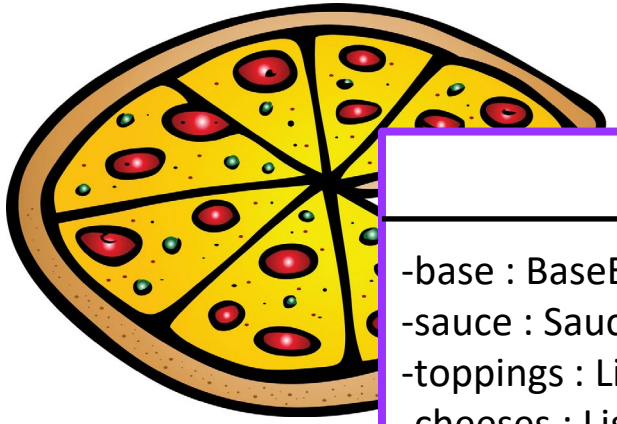


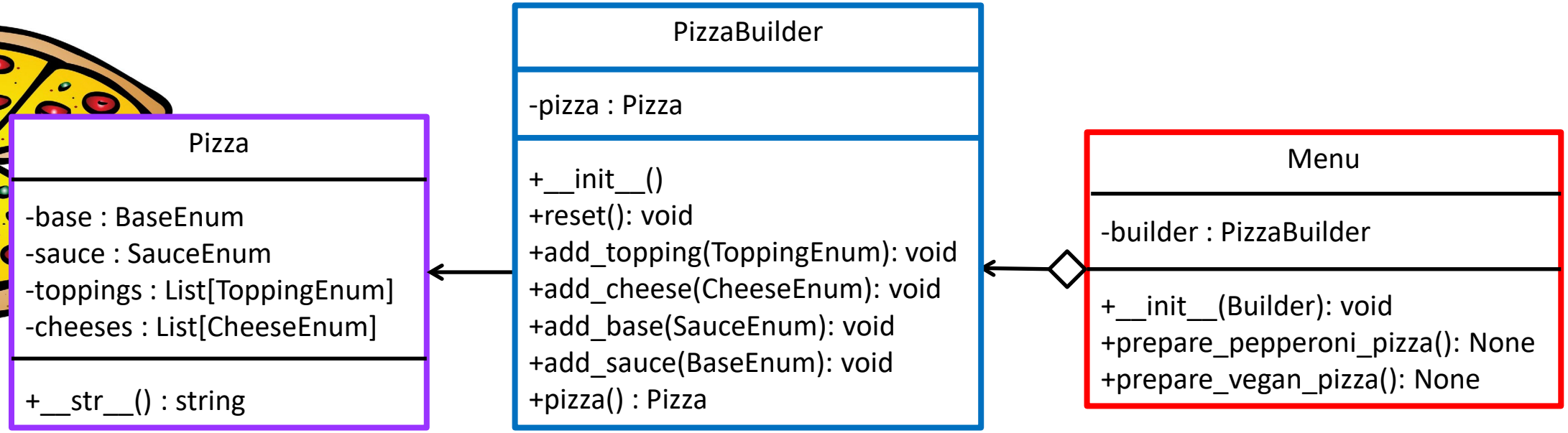
Builder Pattern

Let's look at some code:

bad_pizza.py

builder_sample_code.py





Builder Pattern: Why and When do we use it

- When you want to get rid of ugly init methods in Python or the numerous constructors you may need in other languages

- When a product is built out of many parts that can be mixed and matched. This allows you to build variations while (possibly) avoiding inheritance hierarchies

- Single Responsibility Principle comes to life as we separate construction logic from the product logic. That is, we **decouple** complex **construction logic** from the **product**.

Builder Pattern – Disadvantages

- Code complexity increases as we introduce more classes.
- Does not work well for products that cannot be broken down into independent parts.

Mediator and Dependency Injection

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 13 PART 2

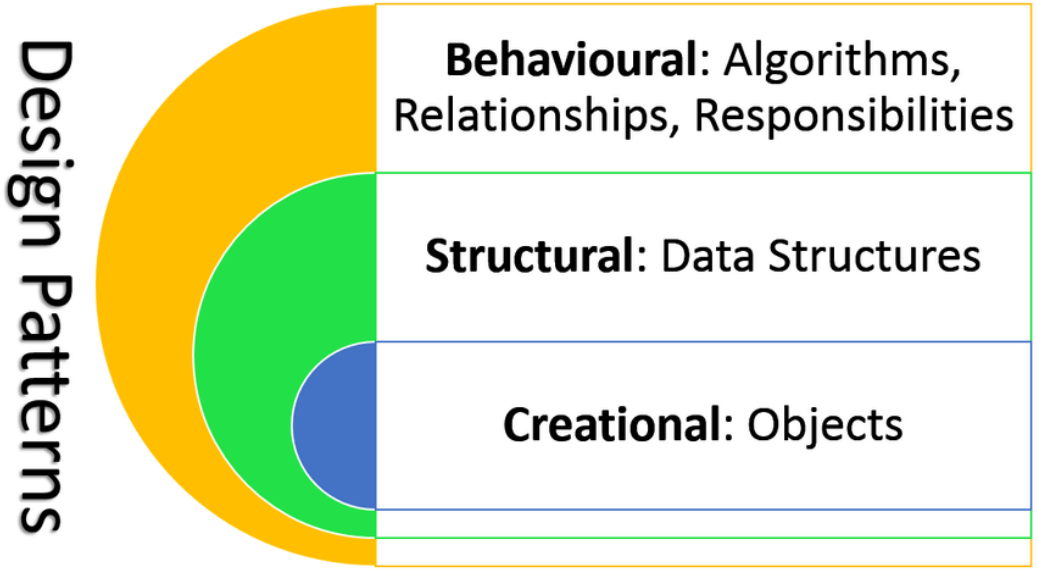
Categorizing Design Patterns

- **Behavioural (We are looking at these!)**

  Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural** How do classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

- **Creational** All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects



Mediator

THE MIDDLE PERSON THAT TAKES CARE OF EVERYTHING SO YOU DON’T HAVE TO.

Mediator – The Problem.

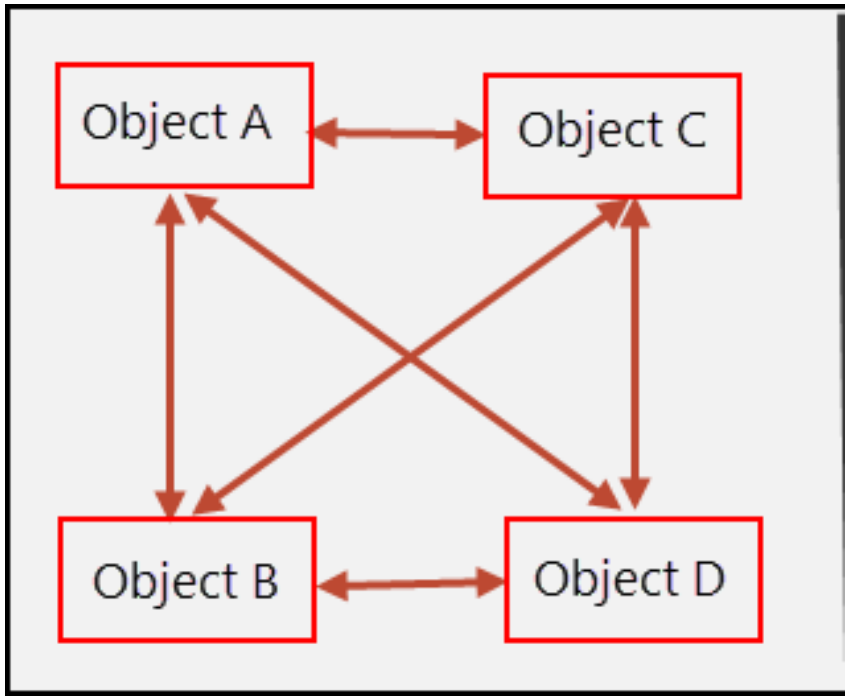
Often our program can be broken down into modular subsystems.

These subsystems are often made up of certain objects (let's call them **components**) that talk and are dependent on each other.

This **communication** between the components facilitates the intended behaviour of the system.

But what happens when this flow of communication becomes **unregulated** and difficult to follow/debug?

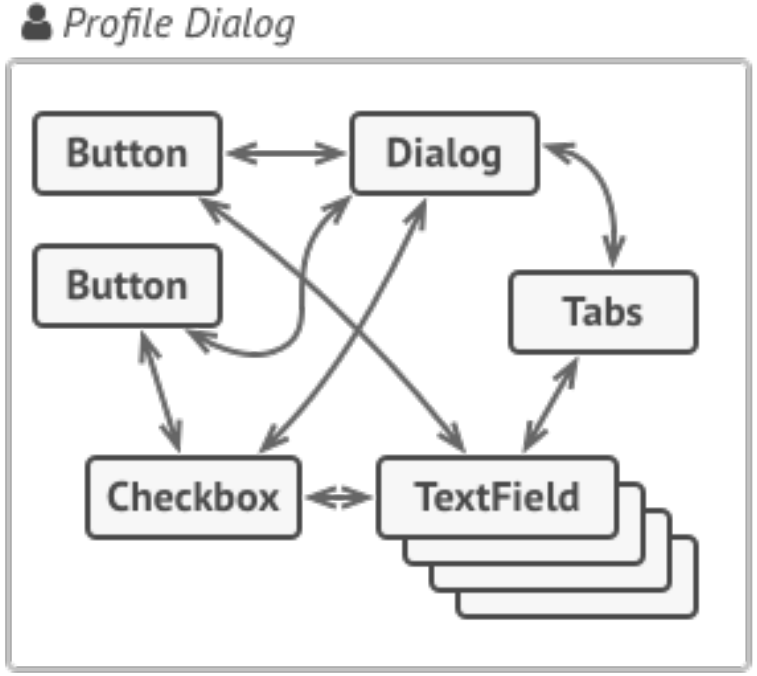
The system becomes **highly coupled** and difficult to maintain.



Mediator – An Example

We often see this in UI systems.

Within a certain **context** (a dialog box, or a screen for example) there are a number of different components that interact with each other.



Mediator – An Example
Person finder

Enter name

Search options

Age

Height

Address

This is a search window to find people, with a number of filters represented as checkboxes. Selecting a checkbox may require that an additional text field be displayed.

Mediator – An Example
Person finder

Jeff

Search options

Age

Height

Address

Entered **name** into search, but want to filter by age and height

Mediator – An Example
Person finder

Jeff

Search options

Age ☒ 99

Selecting age checkbox opens textfield to enter age filter

Height ☐

Address ☐

Mediator – An Example
Person finder

Jeff

Search options

Age ☒ 99

Height ☒ 2000

Address ☐

Selecting height checkbox opens textfield to enter height filter

Mediator – An Example

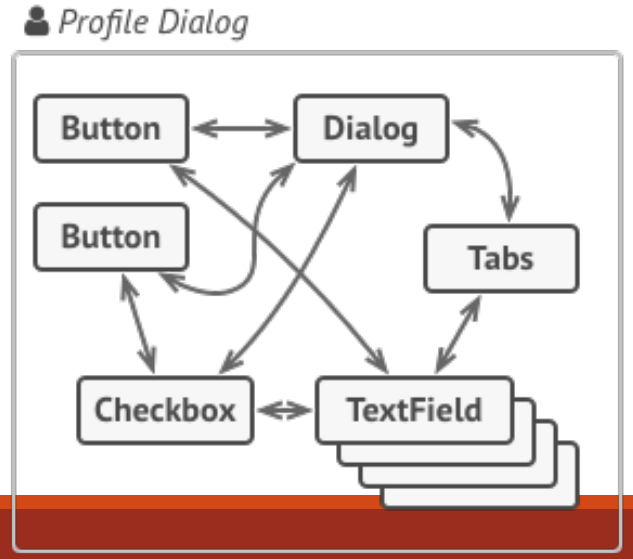
In the previous slides we saw a search window with a number of filters represented as checkboxes. **Selecting a checkbox** may require that an additional text field be displayed.

Below is bad code, it makes it **difficult to re-use the checkbox** in any other screen. Breaks so many SOLID Principles.

Inheriting from the Checkbox class to satisfy this one screen would also be extremely unnecessary and not maintainable.

Age 99

```
class Checkbox:
  def __init__(self)
    self.age_text_field = TextField()
    ..
  if self.selected:
    self.age_text_field.enabled = True
    ..
```

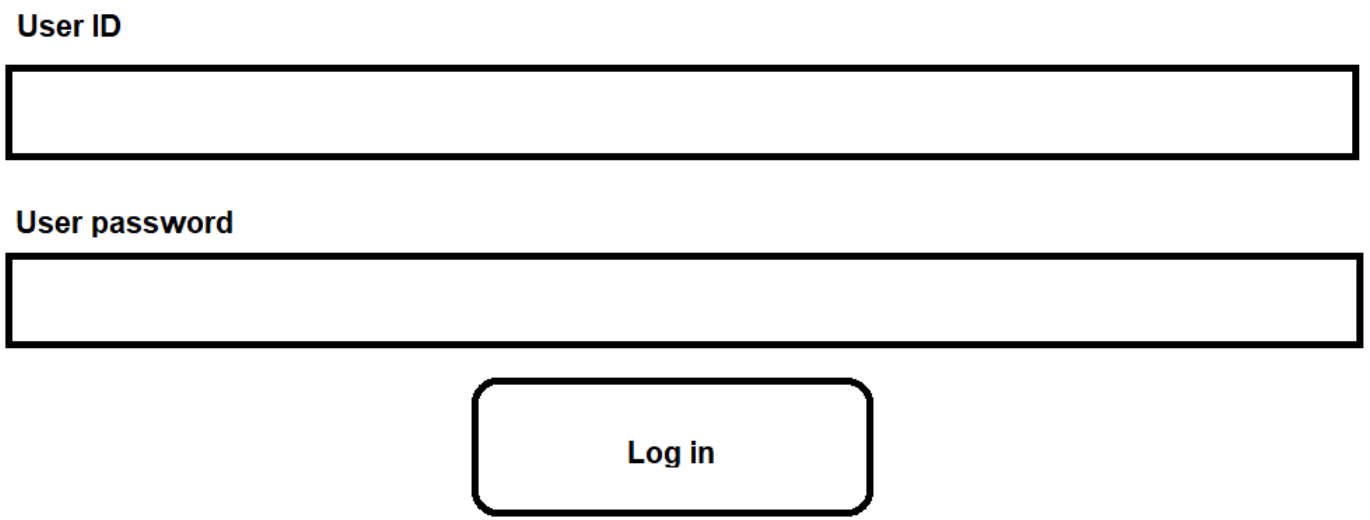


Mediator – An Example

Let's examine a **bad example** of how we may implement the following log in screen.

Notice there are many components:

1. TextLabels
2. Inputfields
    1. User ID field
    2. User password field
3. Screens
4. LogInButton



mediator_problem.py

Mediator – The solution

The Mediator pattern **reduces the dependencies** between a large number of components of a system.

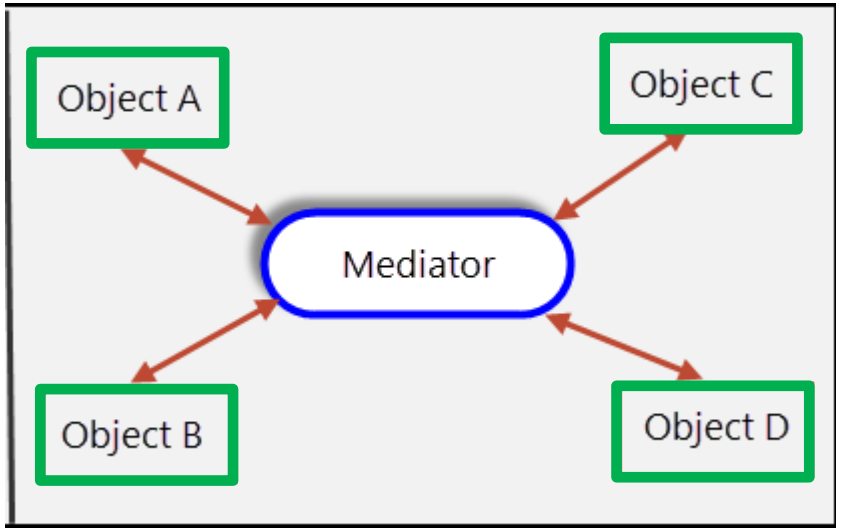
It **restricts communications** between the different components by introducing a middleperson who ***'Mediates'*** the communication between them.

This is very similar to the concept of **Controller classes** that you often come across in code.

The **Mediator class**/entity can be thought of as a formalized structured controller.

The **components** of a system are now unaware of each other and **only depend on a Mediator**.

This relationship is **bidirectional**.



https://www.123rf.com/photo_59304312_traffic-road-traffic-controller-control-transportation-on-street-cop-pointsman-vector-illustration.html

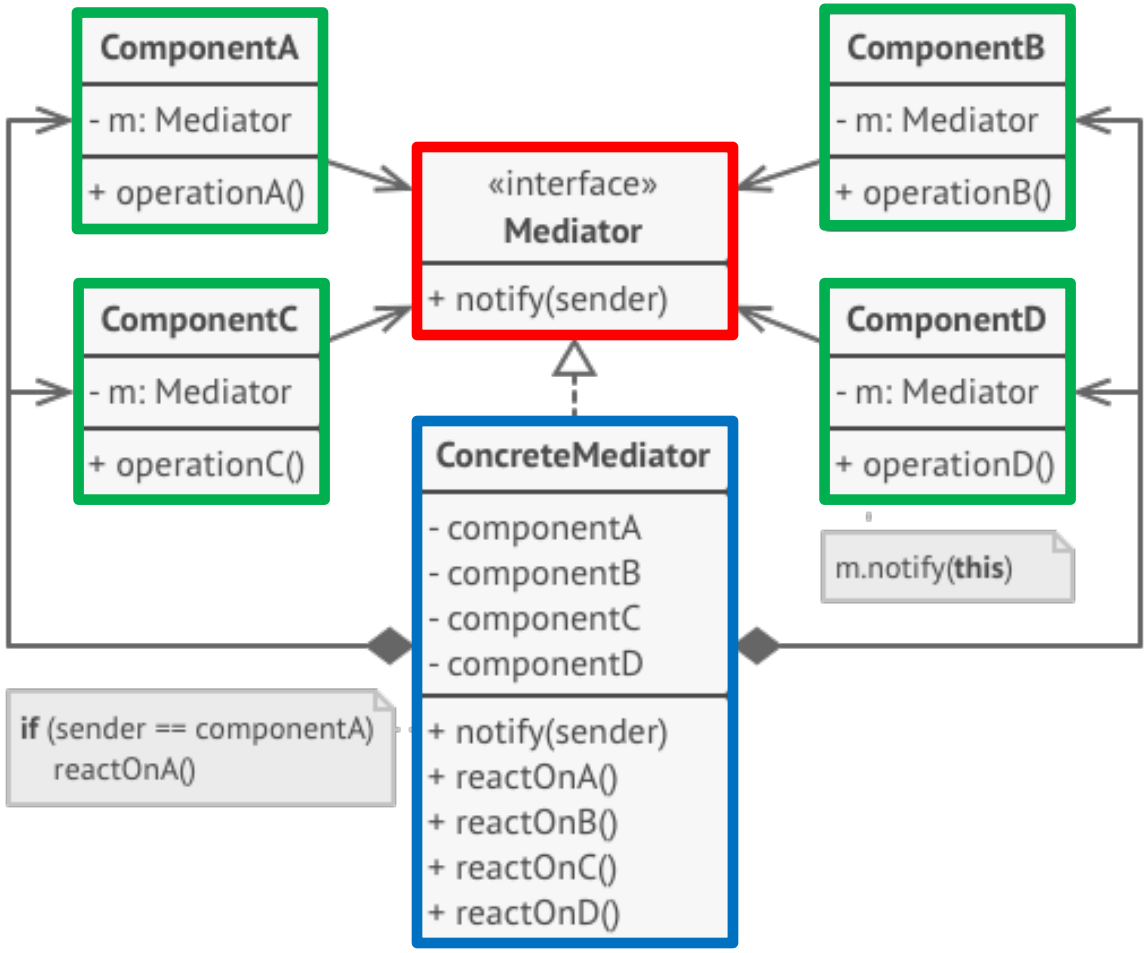
Mediator UML

All Mediators in a system can implement a **common interface**.

Each context or subsystem would have its own **concrete mediator**

By having a common mediator, we can **re-use the components across contexts**.

For example, a button can be used with a LogInMediator and a RegisterUserMediator.



Behavioural – Mediator

Example: Model a traffic light system

TrafficLight can change its color to red, yellow, and green

Button to change TrafficLight's color to red, yellow, green

Car will:

- stop on a red light

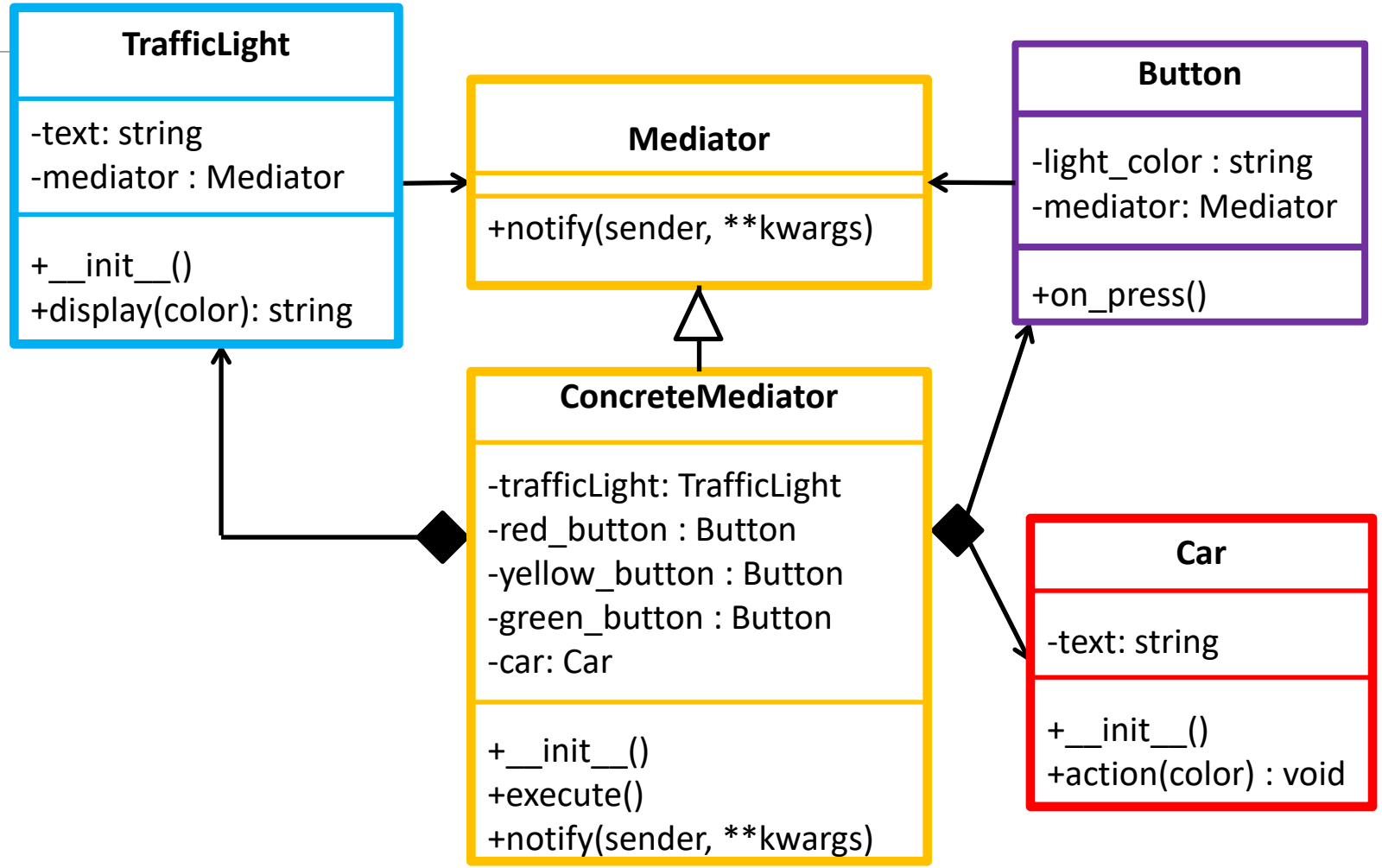
- slow down on a yellow light

- go on a green light

We need all these separate components to communicate with each other, but without coupling them

Have the **Mediator** handle passing messages between all the components

traffic_mediator.py



Some important tidbits.

Mediators don't necessarily have to only be for UI systems

They can be used in other systems where we need **objects or subsystems to talk to each other while being decoupled and unaware of each other**.

Similar to the Observer Pattern, **Mediators** can **cause different parts of a system to react to a component**.

That being said, it is different from an Observer pattern.

- **Mediators** facilitate **bidirectional communication**.
- An **Observer** is more of a **broadcast system** that gets triggered when the state of the observed object changes.

Mediator: Why and When do we use it

- A Mediator should be introduced when the **communication** patterns between different objects **becomes chaotic and highly coupled**.
- Use this pattern when you want a component to be **unaware of other components** but **still be able to communicate** with them.
- A Mediator may be the right choice if you are considering inheritance just to reuse some basic behaviour across contexts
- Implements the Single Responsibility principle. The **responsibility to facilitate communication between objects is extracted into one class**.
- Implements the Open/Closed Principle. We can introduce **new mediators for new contexts without changing the components**.

Mediator – Disadvantages

- The Mediator itself becomes a complex object and an **epicentre for coupling**.

Check out God Objects, this is what is known as an *Anti-Pattern.*

a **God object** is an object that *knows too much* or *does too much*

https://en.wikipedia.org/wiki/God_object

Dependency Injection

WHEN YOU WANT DEPENDENCIES BUT YOU **DON'T WANT TO HARDCODE** THE TYPE OF DEPENDENCY

Dependency Injection

We are going to talk about some general design

Dependency Injection is a:

- Concept
- Technique
- Framework
- A phrase that you may hear often, especially when working with complex systems.

This sounds scary but once you understand the concept it feels obvious.

In fact. You have all written a fairly complex system that implements Dependency Injection already.

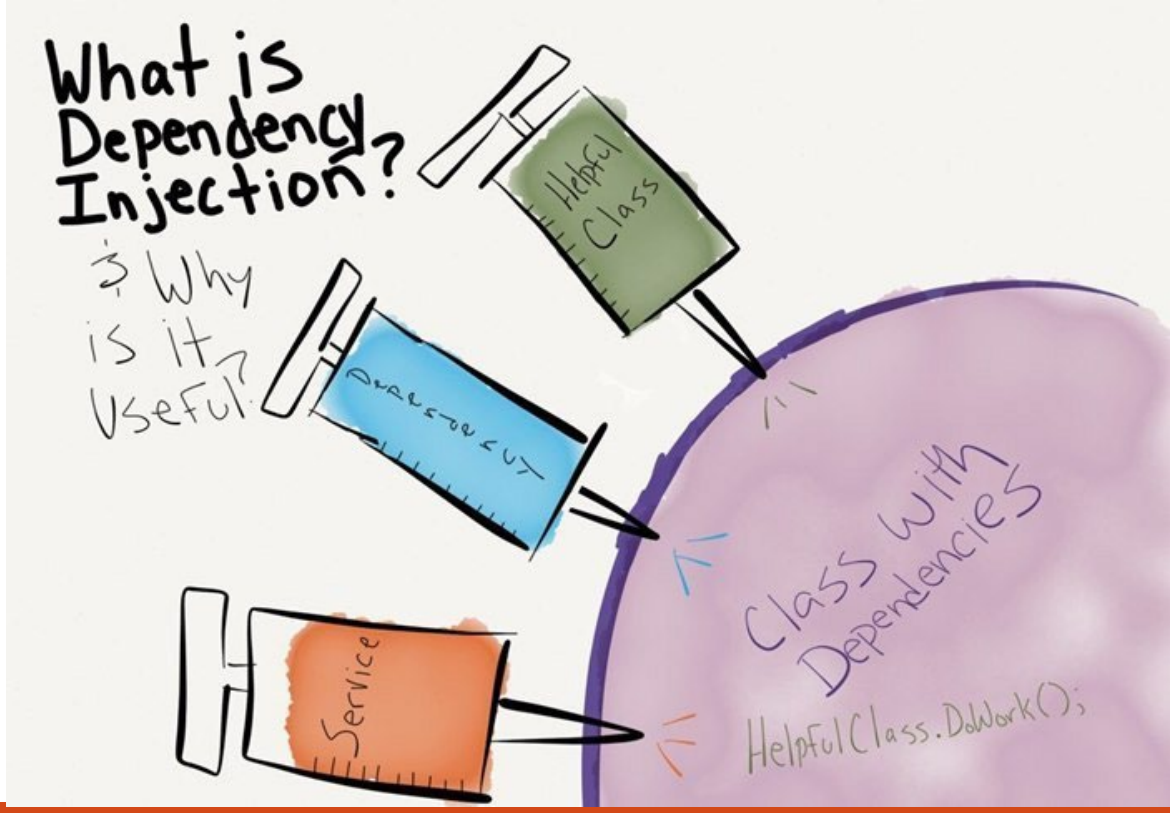
Dependency Injection – Concept

Class A is **dependent** on an **entity B** (a class, a system, an external package, etc.)

Our goal through many design patterns is to control and manage these dependencies.

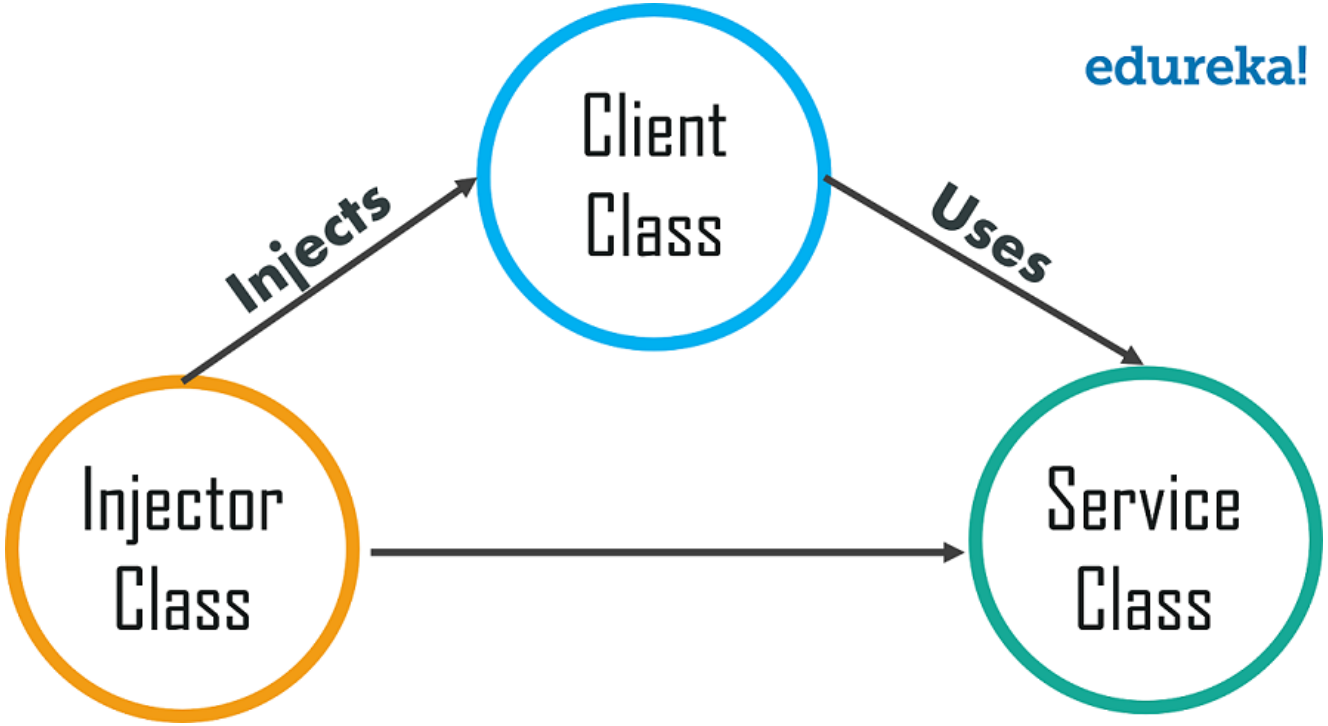
How can we provide a dependency to a dependent class in a way that *reduces coupling*?

Dependency Injection is a concept and a set of software design principles that enable us to write code that can answer this question.



Dependency Injection

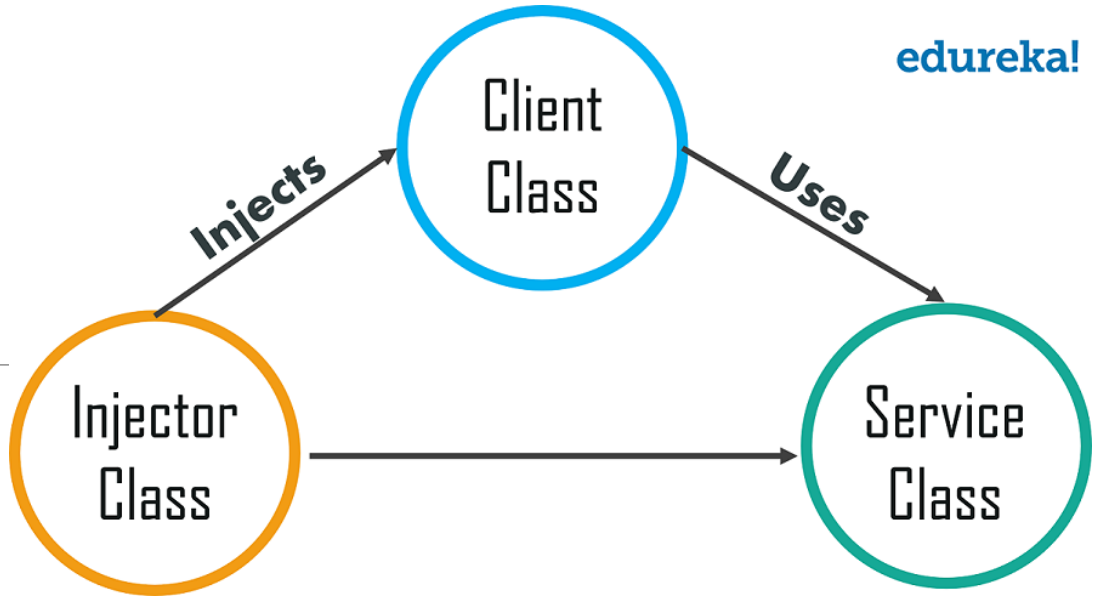
Dependency Injection involves having an external entity (**the Injector**) be responsible for creating (or retrieving) the right dependency for the use case and providing it to the client class.



Dependency Injection involves having an external entity (**the Injector**) be responsible for creating (or retrieving) the right dependency for the use case and providing it to the client class.

To decouple **Class A** from **Class B** and achieve dependency injection:

1. **Class A** should **not** explicitly refer to **Class B**.



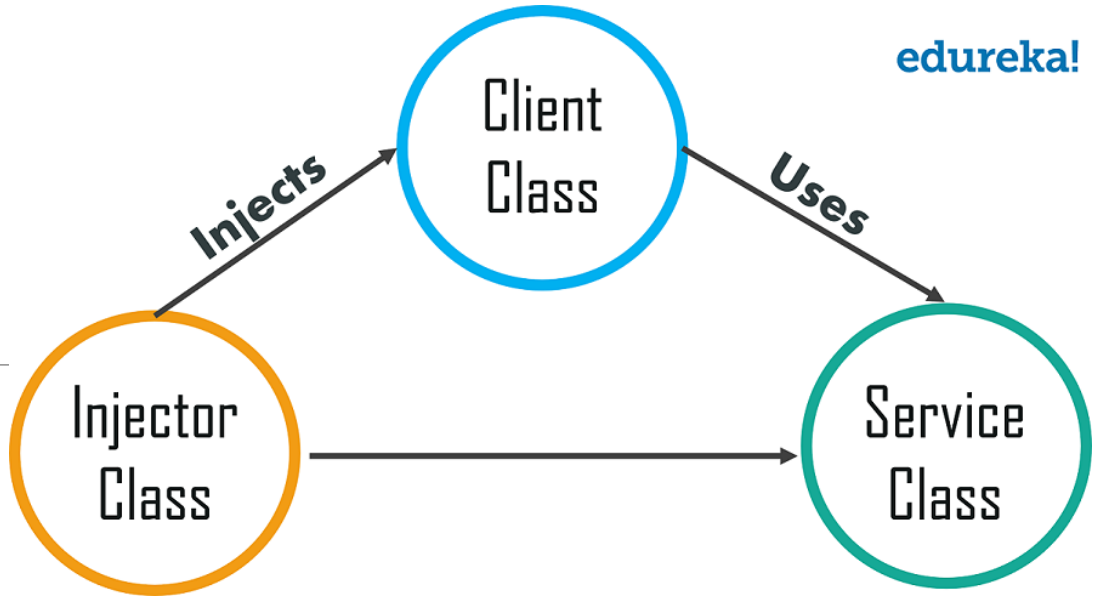

```
class ClassA:
    def __init__(self):
        self.object_b = ClassB()
```

```
class ClassB:
    pass
```

Dependency Injection involves having an external entity (**the Injector**) be responsible for creating (or retrieving) the right dependency for the use case and providing it to the client class.

To decouple **Class A** from **Class B** and achieve dependency injection:

1. **Class A** should **not** explicitly refer to **Class B**.
   - Not mention its type (For languages with static typing)
   - **Refer to an abstraction** instead. Usually a common interface of some sort.



A little better...

```
class ClassA:
    def __init__(self):
        self.abstract_object_b = ClassB()
```

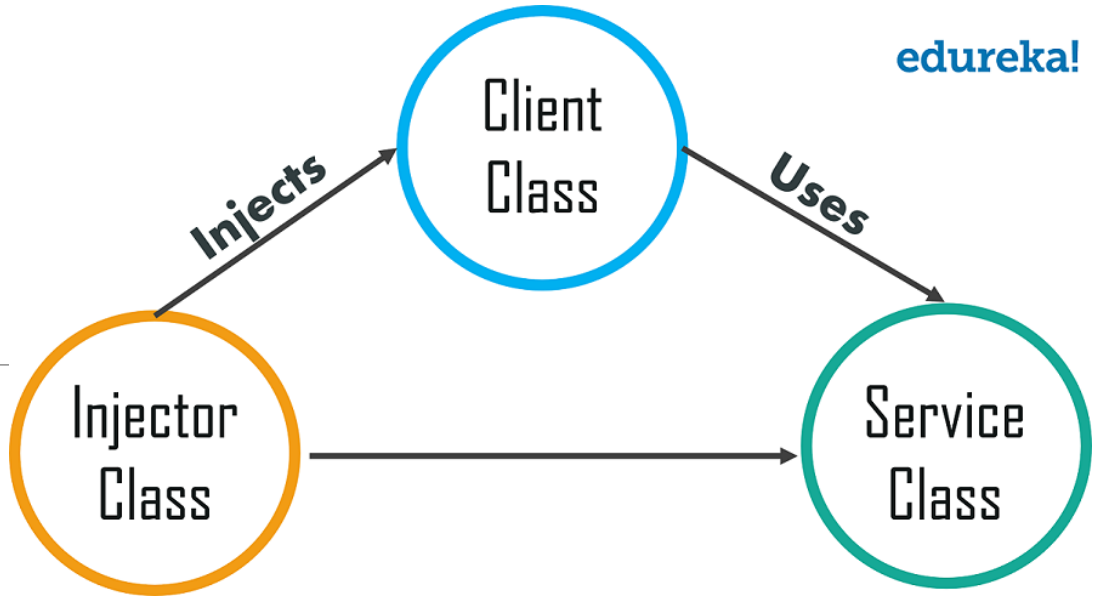
```
class AbstractClassB(abc.ABC):
    pass
```

```
class ClassB(AbstractClassB):
    pass
```

Dependency Injection involves having an external entity (**the Injector**) be responsible for creating (or retrieving) the right dependency for the use case and providing it to the client class.

To decouple **Class A** from **Class B** and achieve dependency injection:

2. **Class A** should **NOT** be responsible for **constructing or importing/retrieving the dependency**




```
class ClassA:
    def __init__(self):
        self.abstract_object_b = ClassB()
```

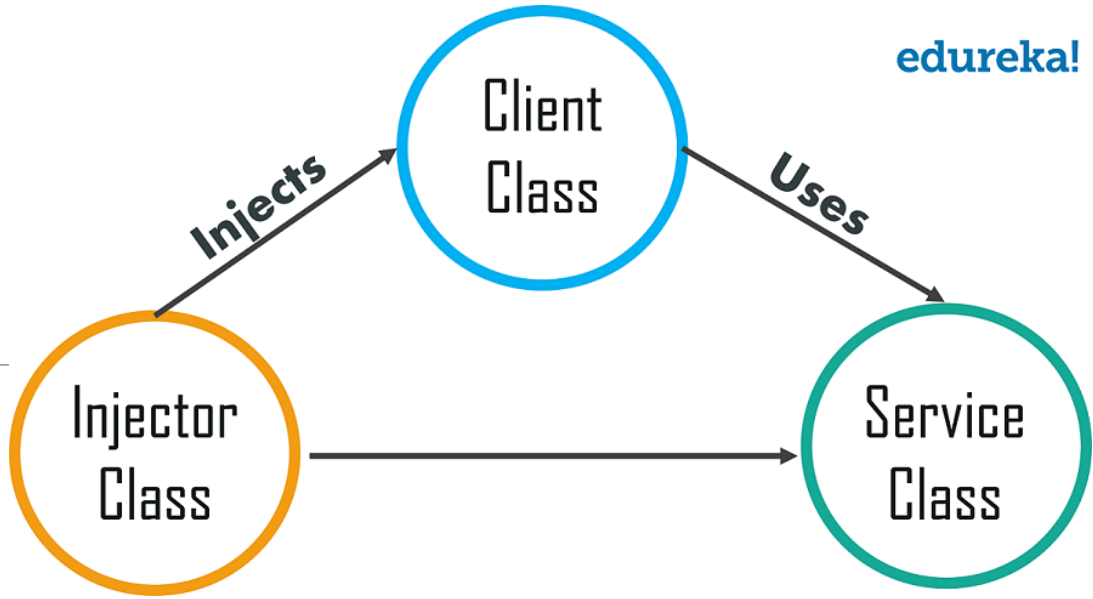
```
class AbstractClassB(abc.ABC):
    pass
```

```
class ClassB(AbstractClassB):
    pass
```

Dependency Injection involves having an external entity (**the Injector**) be responsible for creating (or retrieving) the right dependency for the use case and providing it to the client class.

To decouple **Class A** from **Class B** and achieve dependency injection:

2. **Class A** should **NOT** be responsible for **constructing or importing/retrieving the dependency**




```
class ClassA:
    def __init__(self):
        self.abstract_object_b = None
```

Better… but can't access ClassB

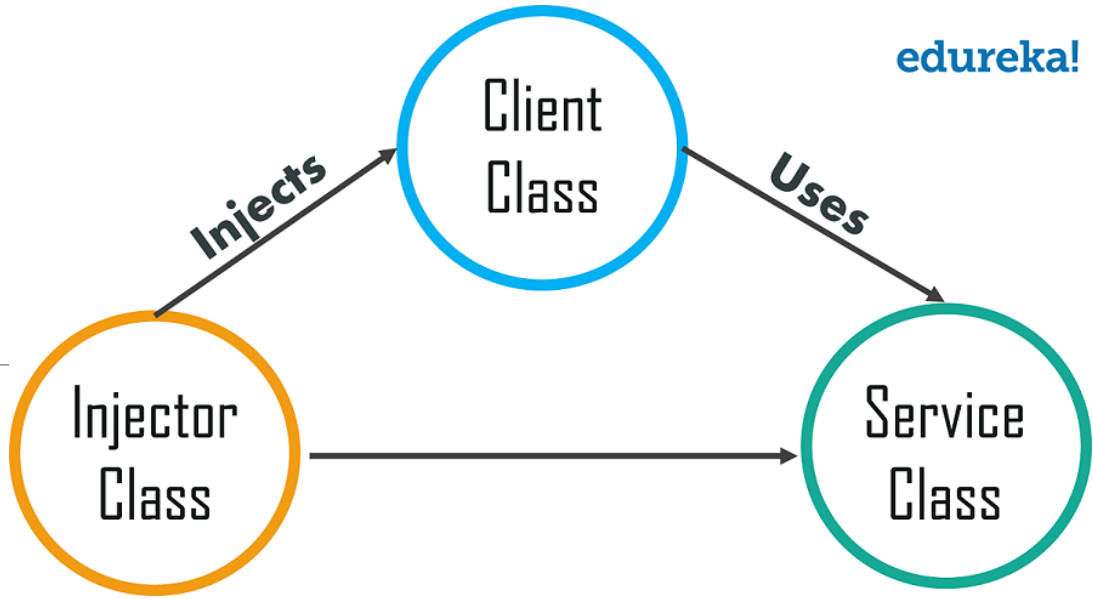
```
class AbstractClassB(abc.ABC):
    pass
```

```
class ClassB(AbstractClassB):
    pass
```

Dependency Injection involves having an external entity (**the Injector**) be responsible for creating (or retrieving) the right dependency for the use case and providing it to the client class.

To decouple **Class A** from **Class B** and achieve dependency injection:

3. **Class B** should be **constructed (or imported/retrieved) by an external entity**. (**The Injector**)




```
class ClassA:
    def __init__(self, object_b : AbstractClassB):
        self.abstract_object_b = object_b
```

BEST!

Uses

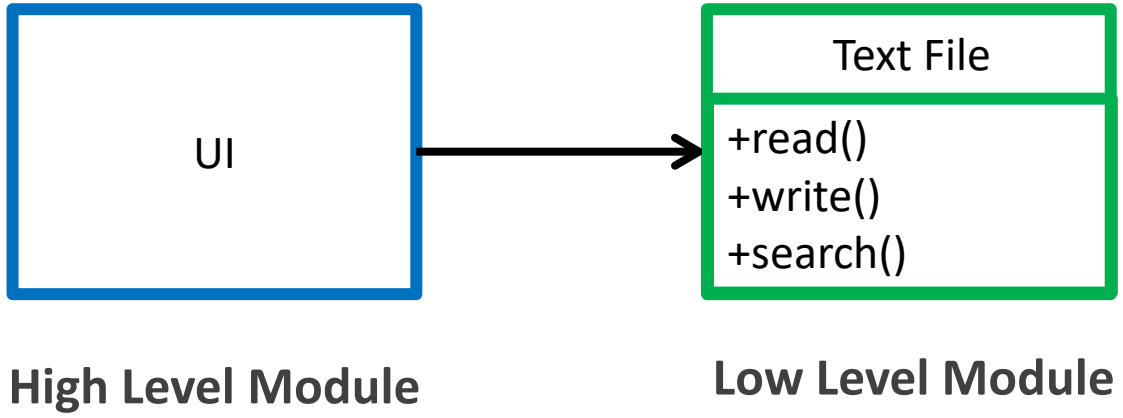
Injects

```
class AbstractClassB(abc.ABC):
    pass
```

```
def main():
    object_b = ClassB()
    class_a = ClassA(object_b)
```

```
class ClassB(AbstractClassB):
    pass
```

RECAP: High Level & Low Level Modules



Remember this slide from way back in Week 3 when we were talking about SOLID principles?

Let's go back to this example. Here we can say that the **UI** is dependent on a **Text File**.

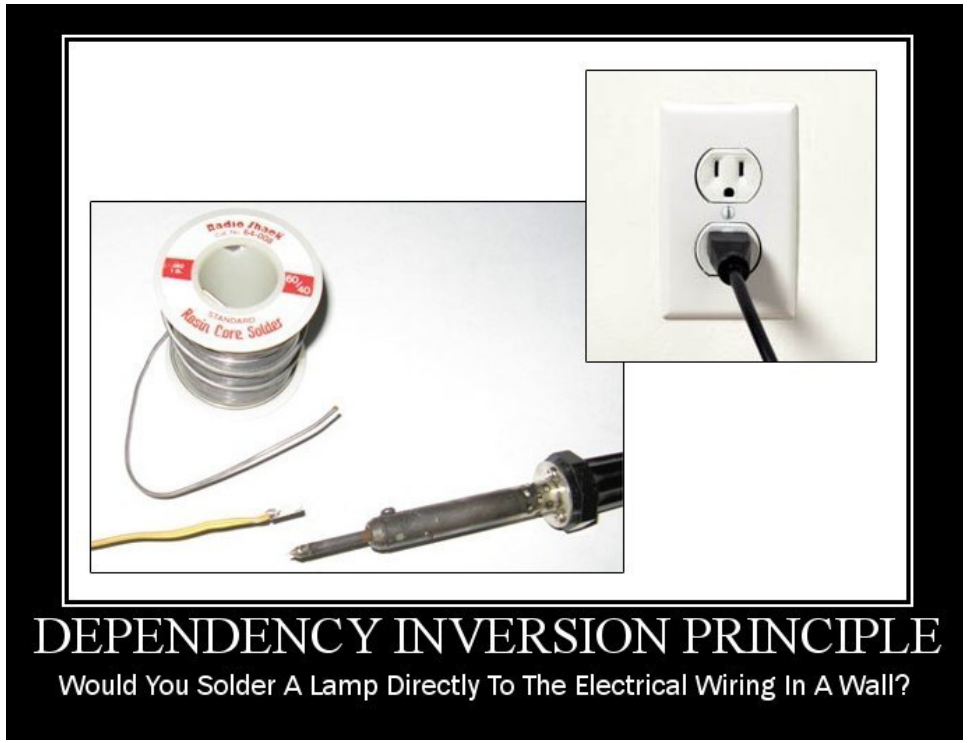
Dependency Inversion Principle Revisited

1. **High-level modules** should not depend on **low-level modules**. **Both should depend on abstractions**.
2. Abstractions should not depend on details. **Details should depend on abstractions**.

We call classes that implement details ‘Concrete Classes’. The Text File class is a concrete class.

Abstract classes declare an interface.

Concrete classes actually contain the code that implements the details of the interface.



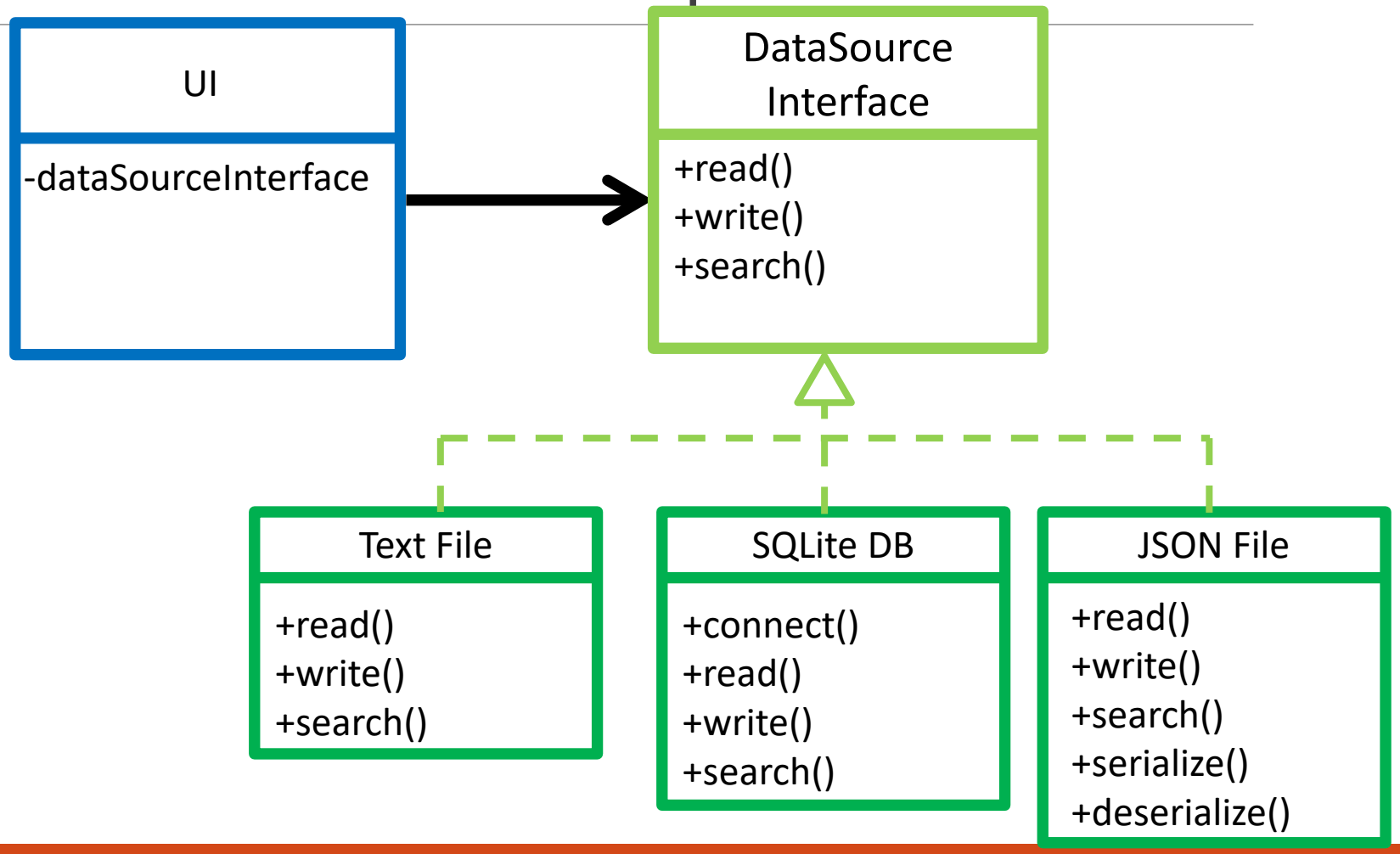
Dependency Inversion Principle Revisited

The **UI class avoids direct references to the SQLiteDB, TextFile or JsonFile classes**.

It is now dependent on a **DataSource** Abstraction/Interface.

So we met one of the requirements of Dependency Injection. In the **UI class** we **no longer have a hardcoded reference to any of the 3 low level modules.**

But if the **UI class** can't instantiate an instance of the **low level modules**, **then how does it have access to one?**



Constructor Dependency Injection

```
class UI:

    def __init__(self, data_source: DataSource):
        self._data_source = data_source  # data source
                                          injected


..
..

def main(self):
    json_file = JSONFile("data.json")
    ui = UI(json_file)
```

We can do this via the **initializer**

By providing the UI class with a reference to a **concrete DataSource** the main method has **injected the dependency**.

The **UI class** is not responsible for importing the **concrete class**. It is done by an external entity.

This is an example of **Constructor Dependency Injection**.

no_dependency_injection.py
dependency_injection.py

Factory Patterns as Dependency Injection

The Factory Pattern is a way of implementing Dependency Injection.

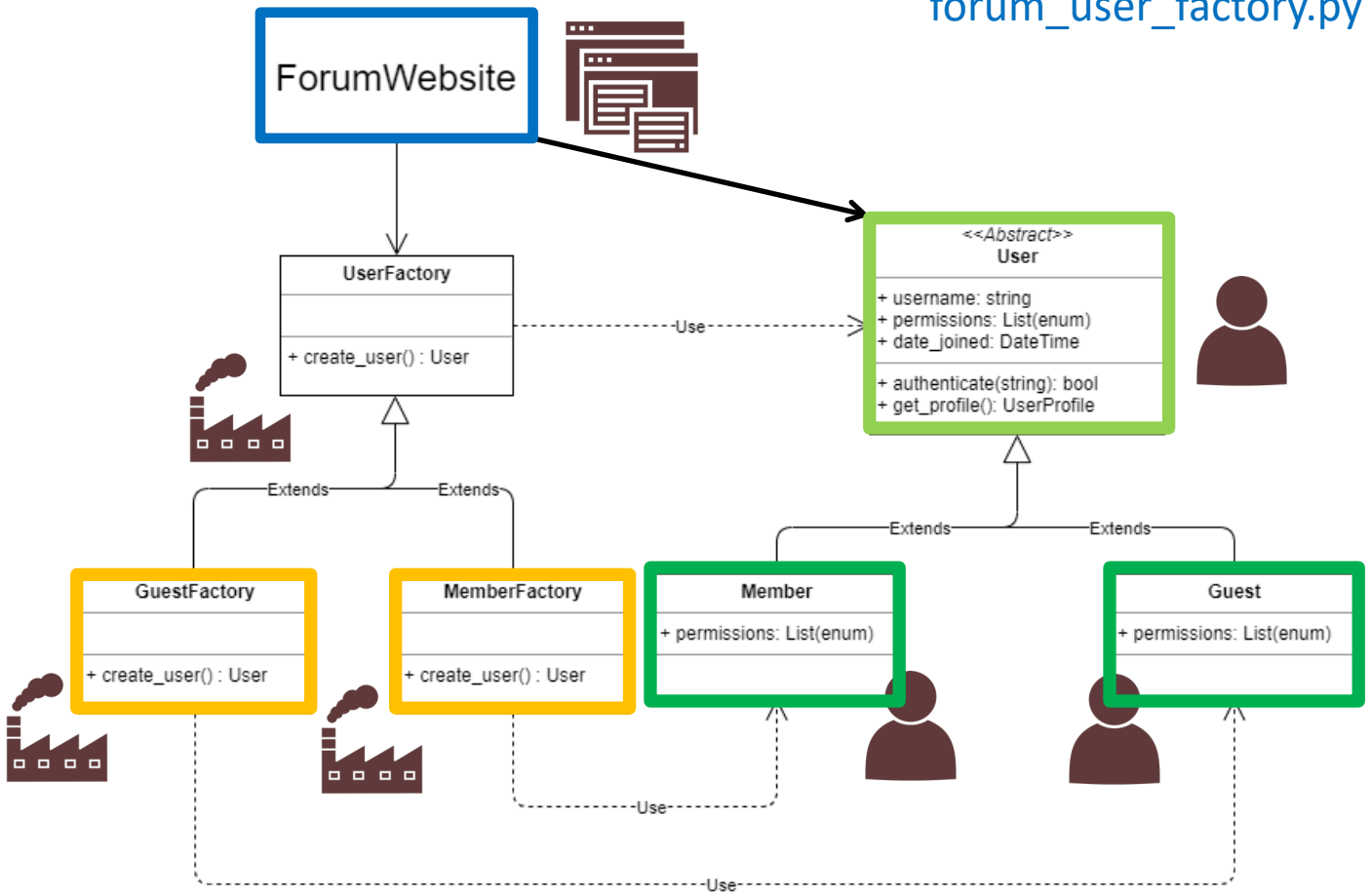
The **client** is dependent on a low level module. We implement the Dependency Inversion Principle by making the **client** **depend on** an **abstract product Interface** (**User**).

The client then needs a way of having access to the User, so we create Factory Classes.

The Factory classes "Inject" the client with the required dependency. **The Factory class is the Injector.**

In this example, the **Forum Website** is dependent on **concrete users**. The **UserFactory injects the ForumWebsite with the right kind of user**.

forum_user_factory.py



Assignment 2 – Dependency Injection

In Assignment 2, the **Store** needed access to all items in its inventory.

- Santa's workshop, RC spider, robot bunny
- Dancing skeleton, reindeer, easter bunny
- Pumpkin toffee, candy canes, crème eggs

By implementing the Abstract Factory, you:

- **Implemented Dependency Inversion Principle**. The **Store** was now only dependent on the **abstract interface of each item**.
- Implemented **Dependency Injection** by creating **factory classes that provided the right item and "details" to the Store**.

The abstract factories were our "**Injectors**".

Dependency Injection is a powerful tool. If done right, it feels like magic.